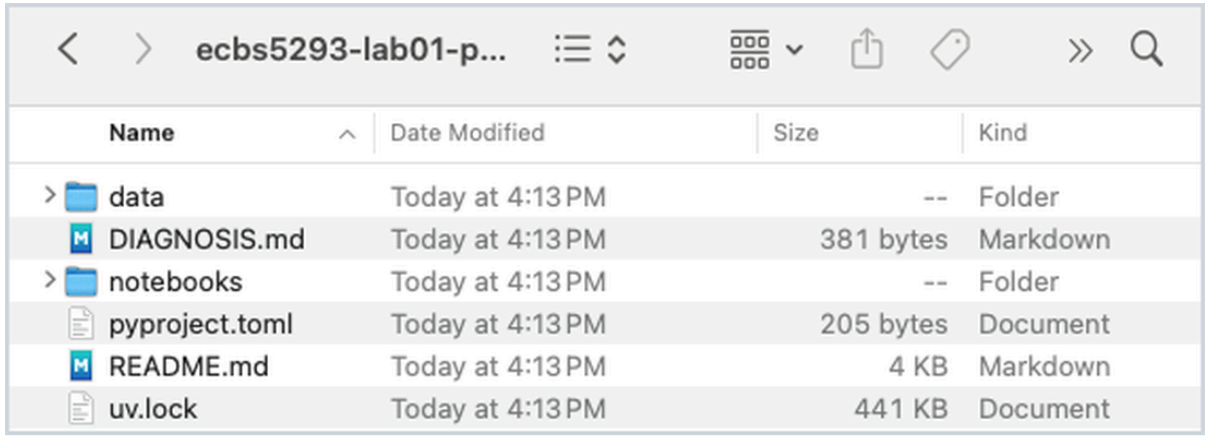
Computing for Analytical Work

Session 1 — Where am I?

Block 2 — The terminal as a way to inspect reality

This block in one sentence

The terminal is not a different computer. It is a precise way to talk to the same computer.



Same folder, same six entries, two views. Finder shows a tidy list. The terminal shows the same list plus exact sizes, times, owners, and permissions.

Agenda

1. Two-minute recap: `pwd`, `ls`, `cd`
2. Shell, prompt, command, output — and the Windows rule
3. Making and moving things: `mkdir`, `cp`, `mv`
4. Looking inside files without Excel: `head`, `tail`, `wc`
5. Running a script with `uv run python …`
6. Recording the commands that worked
7. Git thread: `git diff`. AI thread: ask before you run.
8. Stretch, Lab 2, Homework 1

Two-minute recap: `pwd`, `ls`, `cd`

```
pwd                 # where am I?
ls                  # what is here?
cd notebooks        # go there; every relative path now starts from there
cd ..               # up one level
```

You used all four in Lab 1. If any of them still feels foreign, tell a TA during the lab. We are not re-teaching them; we are building on them.

What a shell is

- The **terminal** is the window. The **shell** is the program inside it that reads what you type.
- The shell shows a **prompt**, waits, you type a **command**, press Enter, it runs it, prints the **output**, and shows the prompt again.
- That loop is the whole thing. Prompt, command, output, repeat.

```
anna@laptop project $ ls data/raw
sales.csv
anna@laptop project $
```

Anatomy of a command

1. **command** the program to run
2. **option (flag)** changes how it behaves: here, a long listing
3. **argument** what to act on: a path, resolved from the cwd

```
anna@laptop project $ ls -l data/raw
total 96
-rw-r--r--  1 anna  staff  48211 Sep  3 10:42 sales.csv
                           size in bytes  last modified
```

evidence, before you open anything

Spaces separate the pieces. A path with a space in it needs quotes: `"My Data/raw"`. Better: no spaces in project names.

Supported shells and the Windows rule

- **macOS / Linux**: the system Terminal (zsh or bash).
- **Windows**: **Git Bash**, installed with Git for Windows. PowerShell and `cmd.exe` are at your own risk; TAs will not debug them in lab.
- Every command in this course works identically in both.

Windows rule: never type bare `python` in Git Bash. It hangs silently. Always pass an argument:

```
python --version
python -c "import sys; print(sys.executable)"
```

Commands that change things

`mkdir`, `cp`, `mv`

`mkdir` — make a folder

```
mkdir output                # one folder, in the cwd
mkdir -p data/processed     # make parents as needed; no error if it exists
ls                          # confirm
```

- Relative to the cwd, like every other path.
- Programs do **not** create folders for you when they write files. A script that writes `output/summary.csv` into a missing `output/` fails. Remember this in the lab.

`cp` — copy

```
cp data/raw/sales.csv data/raw/sales_backup.csv   # file → new file
cp -r data data_backup                            # folder → folder, recursively
```

- Source first, destination second. Both relative to the cwd.
- Copying makes **two** files. Now there are two truths. Before you copy data, ask whether you actually want that.

`mv` — move, which is also rename

```
mv notebooks/sales.csv data/raw/sales.csv   # move to the right place
mv analyze.py analyse.py                    # same folder: that is a rename
```

- One file remains. That is the honest fix for a misplaced file.
- `mv` will **silently overwrite** a file at the destination. Look first: `ls data/raw`.

Predict: `mv` from the wrong place

You are standing in `project/notebooks/`. You type:

```
mv notebooks/sales.csv data/raw/sales.csv
```

What happens? Decide before the next slide. Hands: moves, or fails?

`mv` from the wrong place — what happened

```
mv: notebooks/sales.csv: No such file or directory
```

- Both arguments resolve from the cwd. The shell looked for `notebooks/notebooks/sales.csv`.
- Same rule as Block 1, now with a command that **changes** things.
- Had it succeeded from the wrong place, it would have said nothing. Look, then act.

Commands that only look

`head`, `tail`, `wc`

`head` — the first lines

```
head data/raw/sales_2024.csv        # first 10 lines
head -n 3 data/raw/sales_2024.csv   # first 3
```

```
anna@laptop project $ head -n 3 data/raw/sales_2024.csv
date,region,product,units,unit_price
2024-03-13,North,Doohickey,5,4.75
2024-06-19,North,Widget,33,12.5
```

1. **the header row**
   five names your code must match exactly
2. **the delimiter**
   a comma
3. **the date format**
   ISO: year-month-day
4. **the decimal mark**
   a point, not a comma

You now know the delimiter, the header, the date format, and the column names. Before writing a line of code.

`tail` — the last lines

```
tail data/raw/sales.csv         # last 10 lines
tail -n 2 data/raw/sales.csv    # last 2
```

- Files often go wrong at the end: a truncated download, a trailing "Total" row, a blank line, a footer someone typed by hand.
- `head` and `tail` sample the two **ends**. Together they tell you whether the ends agree. The middle stays unseen: `wc -l` is the next check, and a full pass comes in Session 3.

`wc` — how big is it?

```
wc -l data/raw/sales.csv        # count lines
wc -l data/raw/*.csv            # all CSVs at once
```

```
    1201 data/raw/sales.csv
```

1,201 lines. If the header is one line, every record is one line, and nothing trails at the end: 1,200 rows. Say those assumptions when you subtract; a quoted field with a line break in it breaks the arithmetic. When your dataframe says 900, you have a number to argue with.

Why not just open it in Excel?

- Excel does not show you the file. It shows you its **interpretation** of the file.
- It reinterprets dates, drops leading zeros, converts `1,440.00` by locale, and on save may change the delimiter or encoding.
- Then you click Save, and the file on disk is different from the one you were given.
- `head` shows the bytes. It cannot change them.

Rule for the course: inspect with the terminal; analyse in Python; open in Excel only when you mean to edit.

Predict: a script run from `scripts/`

Two launches. Inside the script, both times, the code opens `data/raw/sales.csv`.

```
cd scripts && uv run python analyze.py            # launch A
cd .. && uv run python scripts/analyze.py         # launch B, back at the project root
```

For each launch, resolve **two** paths from the shell's cwd, and decide whether each exists in the session tree:

1. The **script** path — the thing you typed after `python`.
2. The **data** path — the thing the script opens.

Decide, then the next slide.

Running a script

```
cd /path/to/project                  # the root — where the README says to be
uv run python scripts/analyze.py     # launch B: the script path and every path inside it resolve from here
```

- **Launch A**, from `scripts/`: the script path `analyze.py` resolves to `scripts/analyze.py`, exists. The data path resolves to `scripts/data/raw/sales.csv`, does not.
- **Launch B**, from the root: `scripts/analyze.py` exists, and the data path resolves to `data/raw/sales.csv`, exists. The cwd changed, so both paths changed meaning, and the command had to change to match.
- The script's cwd is **where you ran it from**, not where the `.py` file lives. Same rule as the notebook kernel in Block 1. Output goes wherever the script's relative paths point — from the cwd.

Sidebar: you have been typing `uv run` since setup

- The pre-course checklist had you run `uv sync` and `uv run python -c "import pandas; …"`.
- Lab 1 had you run `uv sync` and pick the `.venv` kernel. Lab 2 is `uv run python scripts/summarize.py`.
- For now it is a **recipe**: type it exactly, from the project root.
- Session 2, Block 3 opens by explaining what it has been doing all along. Until then, do not substitute bare `python`, and do not ask why. Just insist on it.

Recording the commands that worked

Keep a plain text file — `COMMANDS.md`, `notes.txt`, whatever — and paste in the exact lines that worked, in order, with the folder you were in.

```
# from the project root
uv sync
mkdir -p output
uv run python scripts/summarize.py
ls output
```

Not the ones you tried. The ones that **worked**. That file becomes your README's run instructions — and *those* are a homework deliverable.

Git thread — `git diff`: what did I change?

```
git status      # which files changed?
git diff        # what changed inside them?
```

```
--- a/scripts/summarize.py
+++ b/scripts/summarize.py
-OUTPUT = "../output/summary.csv"
+OUTPUT = "output/summary.csv"
```

Lines starting with `-` are what was there. Lines with `+` are what you wrote. Everything else is context.

`git diff` covers **tracked** files only. A file you created is not in it. A file you *moved* shows as a deletion at the old place and nothing at the new one; `git status` lists the new location as *untracked*.

Git thread — why `git diff` matters today

- Before checkoff, run `git diff`. If the diff shows more than your fix, you changed more than you meant to.
- The diff **plus** `git status` are your "what did you change?" answer. Paste both into the diagnosis note; a move is a `deleted` line and an `untracked` line, so say it in words too.
- Windows: if `git diff` shows every line of every file changed, `core.autocrlf` is not set to `input`. Fix it before you continue, or the diff is unreadable.

AI thread — ask before you run

```
I am in Git Bash on Windows, in the root of a Python project.
Before I run it, what does this command do, step by step?

    rm -rf output/*

Is there a version that only deletes CSV files, and asks before each one?
```

- **Explain before executing.** Especially anything with `rm`, `mv`, `-r`, `-f`, or `*`.
- **Ask for a safer version** of a destructive command. `rm -i`, a narrower pattern, a dry run.
- Then read the explanation, and decide.

AI thread — you are responsible for what runs on your machine

- The assistant did not press Enter. You did.
- "AI told me to" is not a diagnosis, not an excuse, and not something you can explain at checkoff.
- AI can help interpret commands and output. **You are responsible for what runs on your machine.**
- Good uses today: explain this command, explain this output, give me a safer version. Bad use: paste a block you cannot explain and run it.

5-minute stand-and-stretch

Stand up. Leave the laptop. Water, window, hallway.

Back in five minutes, then 45 minutes of lab. Homework 1 is on the last slide; it will still be there when you return.

Lab 2 — Run the project without clicking around

`scripts/summarize.py` reads a CSV with the standard library only — no pandas; this lab is about paths and the shell. First inspect the input with `head` and `wc -l`, as the README says. Then follow the README's *Reproduce the failure* block exactly and read what happens. Then the loop: observe → locate → hypothesize → inspect → change → verify. There is more than one thing wrong, and the first fix is not the last. Record the exact commands that worked. `git diff` to see your change. One diagnosis note per thing you fixed. Checkoff.

From the project folder, type this exactly, then the README's *Reproduce the failure* block:

```
uv sync
```

Stretch: make the script robust to the second thing you found, so the next person never meets it. And: why would this also run with bare `python scripts/summarize.py` and no `uv sync`? Hold that for Session 2.

Homework 1 assigned — Files, paths, terminal, and run instructions

A colleague's project that does not run as documented. Make it run, prove the report came from the supplied data, and document how. Expected time: **5–6 hours**. Due the **Friday after this session, 23:59** — slot on Moodle.

Deliverables, on Moodle:

1. The fixed repo with corrected paths. The grader regenerates the output from your run instructions — so show in your note that it did.
2. Run instructions in the README: the exact commands that worked, in order, from the project folder
3. A diagnosis note per failure: symptom, cause, evidence, change, verification — with the sanity counts (rows in, rows loaded, products out) **and the reconciliation**: revenue totalled from the input equals the report's total, to the cent
4. A **60–90 second video walkthrough**: screen recording, run the fixed code, then for every failure you fixed: what broke, why, what you changed, how you verified. No script reading.
5. The zip, made by one command in the README: `uv run python make_submission.py`. No Git needed yet — Git today is `status` and `diff`, and the diff belongs in your note.